SQL SERVER · DATABASE DESIGN PROJECT · ADVANCED LEVEL

Hospital & Clinic Management System

Design and implement a relational database for a multi-department hospital.

01

General Instructions

1. Ensure proper security is in place. Do not give all users direct access to all tables. Use Views, Roles, and GRANT / DENY / REVOKE statements as needed for SQL Server.
2. Design your tables to reflect realistic data. Insert enough sample records so that every task produces a meaningful result — avoid too little or too much data.
3. Extra marks will be given for code that is well-formatted, readable, and properly commented. Use consistent naming conventions throughout.
4. There is no single correct solution. You will be evaluated on how well your solution fits the real-world scenario and how clearly you have reasoned through your design choices.
5. Use `THROW` to stop a procedure or trigger and return a clear error message. Avoid `RAISERROR` — it is the older syntax and has been replaced by `THROW` in modern SQL Server.
6. All procedures, triggers, functions, and standalone queries must be included in one single `.sql` file, clearly separated with comment headers.

7 Wrap procedure and function bodies in `TRY...CATCH` blocks where appropriate.

02

Scenario Overview

You are building the backend database for **MediCare+**, a private hospital with several departments. Read the description carefully — your database design must support everything described here.

HOW THE HOSPITAL OPERATES

The hospital is divided into departments such as Cardiology, Orthopaedics, and Pathology. Each department has a head doctor. Doctors belong to one department, have a specialisation, and charge their own consultation fee. Each doctor also has a weekly schedule that defines which days and time slots they are available.

Patients register with the hospital. Some patients have insurance — the policy covers a percentage of their medicine and lab costs, up to a yearly maximum. When a patient books an appointment with a doctor, it is recorded with a date, time slot, and a status: Scheduled, Completed, or Cancelled.

Once an appointment is completed, the doctor creates a medical record with the diagnosis and treatment plan. The doctor may prescribe medicines — each prescription links a medicine with a dosage, duration, and quantity. Separately, the doctor may order lab tests; results are recorded later and can be flagged as abnormal.

After the appointment is complete, a bill is generated. The bill covers the consultation fee, medicine costs, and lab test costs. Insurance discount (if any) is applied first, then GST is added at different rates for different charge types. The bill shows the final payable amount and a payment status.

BUSINESS RULES

These rules must be enforced in your database — through constraints, triggers, or procedures — not just in your application logic.

> A doctor cannot have two appointments at the same date and time.

> A bill can only be generated for an appointment with the status *Completed*.

> GST rates differ by charge type: 0% on consultation, 5% on medicines, and 12% on lab tests.

> Insurance discount (where applicable) applies to medicine and lab costs only — not to the consultation fee. It is applied before GST is calculated.

> If a lab test result is flagged as abnormal, the linked medical record must be updated to require a follow-up.

> A patient's total unpaid bills must not exceed Rs. 2,00,000 at any point.

> **Your database design is your own.** The scenario describes what the system must support. It is your job to decide which tables to create, what columns they need, and how they relate to each other. You will be judged on how well your design fits the described system.

03

Tasks

Complete all 12 tasks below. Unless stated otherwise, all procedures and functions must accept parameters, handle invalid input gracefully, and raise a descriptive error where needed.

SECTION A — STORED PROCEDURES

A1 Monthly Department Report — EASY

Create a stored procedure that accepts a month and year as parameters and returns a report showing each department, the number of appointments in that period, the number of unique patients seen, and the total consultation revenue. Departments with no appointments in that period must still appear in the result.

A2 Patient Billing Statement — MEDIUM

Create a stored procedure that accepts a Patient ID and returns a full billing history for that patient. For each completed appointment, show the appointment date, doctor name, consultation charge, total medicine cost, total lab cost, insurance discount applied, GST charged, and final amount payable. Include a grand total row at the end. Raise an error if the Patient ID does not exist.

A3 Doctor Performance Report — MEDIUM

Create a stored procedure that returns a performance summary for all active doctors. For each doctor, show their total appointments, total completed appointments, completion rate as a percentage, total revenue generated, and number of unique diagnoses recorded. The procedure must accept a minimum appointment count — only doctors who have handled at least that many appointments should appear. Order results by revenue, highest first.

A4 Medicines Never Prescribed — Two Ways — MEDIUM

Create a stored procedure that returns all medicines that have never appeared in any prescription. Solve this in two different ways within the same procedure: once using a subquery approach, and once using a SET operation. Both must return the same result. Label each approach clearly with a comment.

A5 Monthly Revenue vs. Target — MEDIUM–HARD

The hospital's monthly revenue target is Rs. 5,00,000. Create a stored procedure that returns one row per calendar month showing: the month and year, total revenue collected, whether the target was met (Yes or No), and the surplus or deficit amount. Also include a final summary showing how many months met the target and how many did not.

SECTION B — TRIGGERS

B1 Prevent Doctor Double-Booking — MEDIUM

Create a trigger on the Appointments table that fires when a new appointment is inserted. If the doctor already has a Scheduled or Completed appointment at the same date and time, the trigger must roll back the insert and raise a clear error message identifying the doctor and the conflicting time slot.

B2 Auto-Generate Bill on Completion — HARD

Create a trigger on the Appointments table that fires when a row is updated. When the status is changed to *Completed*, the trigger must automatically insert a fully calculated bill into the Billing table. The bill must include the consultation fee, total medicine charges, total lab charges, the correct insurance discount (if any), GST at the appropriate rates for each charge type, and the final payable amount. Raise an error if a bill already exists for that appointment.

B3 Flag Follow-Up on Abnormal Lab Result — EASY–MEDIUM

Create a trigger on the lab orders table that fires on both INSERT and UPDATE. When a result is marked as abnormal, the trigger must automatically update the linked medical record to require a follow-up. If no medical record exists yet for that appointment, the trigger should raise a warning message but must *not* roll back — the lab order must still be saved.

SECTION C — USER-DEFINED FUNCTIONS

C1 Patient Age Calculator — EASY

Create a scalar function called `fn_GetPatientAge` that accepts a Patient ID and returns the patient's current age in completed years as an integer. Return NULL if the Patient ID does not exist. Demonstrate its use in a SELECT query that lists all patients with their calculated age.

C2 Net Bill Calculator — MEDIUM

Create a scalar function called `fn_CalculateNetBill` that accepts a consultation charge, medicine charge, lab charge, and insurance coverage percentage (pass 0 if no insurance applies) and returns the final payable amount after applying the insurance discount and the correct GST rates. Demonstrate its use by calling it in a SELECT on your billing data and verifying the result matches your stored totals.

SECTION D — ADVANCED STANDALONE QUERIES

Write these as standalone queries, not inside procedures. Label each with a comment showing the task number and a short description.

D1 Top 3 Doctors per Department by Revenue — HARD

Write a query that returns the top 3 revenue-generating doctors within each department. Show the department name, doctor name, total revenue, and their rank within the department. Doctors with no completed appointments must not appear.

D2 Running Monthly Revenue Total

MEDIUM–HARD

Write a query that shows, for each calendar month, the total revenue collected and a running cumulative total from the earliest month in the data up to that month. Order the results chronologically and display the month in a readable format — for example, *Jan 2024*.

04

Security Requirements

Create the following database roles and assign permissions using GRANT, DENY, and REVOKE. No role should have broader access than what is listed. Where a role needs patient information but should not see all columns, create a View that exposes only the required fields — grant access to the View, not the base table.

ROLE	REPRESENTS	ALLOWED ACCESS
`db_receptionist`	Front-desk / registration staff	SELECT and INSERT on patient and appointment data only. No access to billing, medical records, prescriptions, or lab orders.
`db_doctor`	Treating doctors	SELECT on patient and appointment data. INSERT and UPDATE on medical records, prescriptions, and lab orders. No access to billing.
`db_lab_tech`	Laboratory technicians	SELECT on lab test reference data and lab orders. UPDATE on lab orders limited to result fields only. No access to patient personal information or billing.
`db_billing`	Billing and accounts staff	

ROLE	REPRESENTS	ALLOWED ACCESS
		SELECT, INSERT, and UPDATE on billing data. Read-only access to a restricted patient View (name and insurance details only — no date of birth, phone, or address). No access to medical records or prescriptions.
`db_admin`	Hospital IT administrator	Full access to all tables, views, procedures, triggers, and functions.

05

Deliverables & Marking

WHAT TO SUBMIT

1. **ER Diagram** — showing all your tables, primary keys, foreign keys, and the cardinality of every relationship (1:1, 1:N, M:N). A hand-drawn diagram is accepted but must be neat and fully legible.
2. **schema_and_tasks.sql** — one SQL Server script containing: all CREATE TABLE statements in correct dependency order, all INSERT statements, role and permission statements, view definitions, UDFs, stored procedures, triggers, and Section D standalone queries. Each section must be clearly labelled with a comment header.
3. **Output Evidence** — for each task (A1 through D2), a screenshot showing the output when your code is run against your sample data. Label each screenshot with the task number.

MARKING SCHEME

COMPONENT	MARKS	BASIS
Database Design — tables, relationships, and constraints	20	

COMPONENT	MARKS	BASIS
		Correctness, completeness, appropriate constraints
ER Diagram	10	Must match your actual implemented schema
Stored Procedures — Section A (5 tasks × 4 marks)	20	Correctness, parameter handling, error messages
Triggers — Section B (3 tasks × 6 marks)	18	Logic, ROLLBACK handling, meaningful error messages
User-Defined Functions — Section C (2 tasks × 5 marks)	10	Correctness and demonstrated use in a query
Advanced Queries — Section D (2 tasks × 5 marks)	10	Correct use of window functions, readable output
Security — roles, views, and permission statements	8	All 5 roles correctly created with appropriate access

COMPONENT	MARKS	BASIS
Code Quality — formatting, comments, naming, TRY…CATCH	**4**	Readability and professionalism of the .sql file
Bonus — creative schema design, CTE usage, window functions in procedures	*+5*	*At instructor's discretion*
Total	100 + 5 bonus	